TEII

Tecnologías Específicas de la Ingeniería Informática

Extensión de Python: cómputo científico e interfaz con C

Librerías numpy, matplotlib y ctypes

Pedro Enrique López de Teruel Alcolea
Departamento de Ingeniería y Tecnología de Computadores
pedroe@um.es

Grado en Ingeniería Informática

Curso 2018/19

Facultad de Informática

UNIVERSIDAD DE MURCIA

Índice

1. Ampliación de Python
   - Algunas generalidades
   - Uso de librerías
   - Excepciones
   - Más sobre funciones
   - Orientación a objetos
2. Numpy
   - Introducción
   - Fundamentos
   - Indexado, *slicing* y vistas
   - Tipos de datos
   - Operaciones sobre arrays
   - Broadcasting
   - Subpaquetes
3. Matplotlib
   - Fundamentos y gráficos básicos
   - Control de ventanas y GUI
   - Gráficos múltiples
   - *Customization*
   - Orientación a objetos
4. Ctypes
   - Introducción
   - Librería externa en C
   - *Wrapper* de Python
   - Uso desde Python
   - Depuración con `gdb`

Recordatorio de características de Python

- **Open Source**.
- **Alto nivel**:
  - ◇ Simple y minimalista (y por tanto, fácil de aprender).
  - ◇ Tipado dinámico.
  - ◇ Manejo memoria automático.
- **Portable**:
  - ◇ Linux, Windows, Macintosh, ...
  - ◇ P. e., disponible también para Raspberry Pi
- **Interpretado** (no compilado).
- **Orientado a objeto:**
  - ◇ Aunque puede usarse también de modo simplemente imperativo.
- **Extensible con C/C+** fácilmente.
- **Extenso conjunto de librerías** disponibles:
  - ◇ GUI, documentación, threads, bases de datos, web, XML, HTTP, juegos, multimedia, criptografía, proceso imagen, cómputo científico, visión por computador, machine learning...

Python 2.x *vs* Python 3.x

- Python está aún **en transición** de la versión 2 (hasta 2020) a la versión 3 (recomendada).
- Usaremos ya la versión de Python 3, en concreto la **3.6**.
- Usaremos la implementación **Anaconda**:
  - ◇ Mantenimiento de paquetes con mayor facilidad.
- Ojo, hay algunas (pequeñas) **diferencias sintácticas** entre Python 2 y Python 3:
  - ◇ Existen herramientas automatizadas de traducción ...
  - ◇ ... Pero nosotros no tendremos problema porque partimos directamente de Python 3.

Entorno de trabajo

- **Spyder**:
  - ◇ IDE con facilidades de edición / depuración / testeo.
  - ◇ Integra `ipython` (intérprete enriquecido).
  - ◇ Inspector de variables.
  - ◇ Ayuda rápida.
  - ◇ Modelo de código:
    - · Corrector de sintaxis.
    - · Compleción de código.
  - ◇ Análisis de eficiencia (*profiling*).
- Otra alternativas muy populares:
  - ◇ **PyCharm** (*Free Community*): IDE
  - ◇ **Jupyter** (ejemplo *Google Colab*): Entorno interactivo de programación / generación de documentos técnicos dinámicos; muy usado en investigación.

Ejecutar un script Python

- Para iniciar un intérprete de python, teclear en un terminal `python`, o mejor aún, `ipython`. También puede estar integrado en el IDE (spyder, PyCharm). Se sale con *Ctrl-D*.
- Si hemos escrito un *script* de Python llamado `filename.py`, se puede ejecutar así:

```
$ python filename.py
```

- Aunque en Linux se puede hacer al script directamente ejecutable colocando esta línea arriba del todo:

```
#!/usr/bin/env python
```

- También debe haberse hecho previamente ejecutable el archivo, ejecutando el comando:

```
$ chmod +x filename.py
```

Iteraciones sobre listas

- **`enumerate`** es el modo más conveniente de iterar sobre una lista manteniendo el índice:

Con enumerate:

```
cuadrados = [0, 1, 4 ,9, 16, 25]
for i, val in enumerate(cuadrados):
    print(i,val)   # 0 0   1 1   2 4   3 9   4 16   5 25
```

- Es mucho más limpio, conciso y conveniente que:

Sin enumerate:

```
cuadrados = [0, 1, 4 ,9, 16, 25]
i = 0
for val in cuadrados:
    print(i, val)  # Ídem
    i = i+1
```

Importación de librerías

- La cláusula `import` se usa para cargar un módulo (librería, u otro módulo fuente).
- Después se puede acceder a sus funciones, variables, clases, etc., cualificación usando nombre de librería + '.'
- También puede cambiarse el nombre de la misma al importar la librería (para abreviar):

Sin renombrar

```
import math

print(math.cos(math.pi)/2)
```

Renombrando

```
import numpy as np

print(np.array([[1,2],[3,4]]))
```

Importación de librerías

- También se pueden importar sólo funciones, variables, clases, etc. concretas.
- Incluso se pueden importar todos los contenidos de la librería en el espacio de nombres actual.
- Se evita así la necesidad de cualificación.

Importando algo concreto

```
from math import pi, cos

print(cos(pi))
```

Importando todo

```
from math import *
print(cos(pi))
```

- Aunque esto último (`from` *`library`* `import *`) no está muy recomendado, ya que se "llena demasiado" (namespace pollution) el espacio de nombres.

Excepciones

- Si una instrucción (o bloque de ellas) genera una excepción (provocada por cualquier tipo de error), ésta abortará el programa, a no ser que se capture:

Captura de una excepción general

```
while True:
    try:
        x = int(input("Introduzca un número:"))
        break
    except:
        print("Número inválido, inténtelo otra vez.")
```

Captura de una excepción específica

```
    try:
        [...]
    except ValueError:
        print("Número inválido, inténtelo otra vez.")
```

Excepciones

- Se puede también ser más selectivo en la captura de excepciones, obtener información de ellas, etc.:

Diversas posibles excepciones

```
import sys
try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except IOError as e: # Excepción específica, que nombramos
    print('Error de E/S({0}): {1}'.format(e.errno, e.
    strerror))
except ValueError:   # Excepción específica
    print('No se pudo convertir el dato a tipo entero')
except:              # Cualquier otro error (default)
    print('Error inesperado: ', sys.exc_info()[0])
    raise  # Vuelve a levantar la excepción
```

Excepciones

- Otro ejemplo más, usando la instrucción **pass** (que esencialmente sirve para "no hacer nada"), para ignorar varios tipos de excepción:

Ignorar excepciones

```
import sys
i = -1    # Valor por defecto si algo falla
try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
except (IOError, ValueError):    # Excepciones a ignorar
    pass
```

Más sobre funciones

- En python las funciones son objetos normales ("ciudadanos de primera clase").
- Como tales, puden asignarse a variables, pasar como argumentos, etc.:

Funciones *de primera clase*

```
def cuadrado(x):
    return x**2
def cubo(x):
    return x**3
def operar(f, y):
    return f(y)

print(operar(cuadrado, 4)) # Imprime 16
fun = cubo
print(operar(fun, 4))      # Imprime 64
```

Funciones lambda

- Son **funciones anónimas**.
- Se definen en el mismo lugar en que se usan, sin darles nombre explícitamente:

Funciones *anónimas*

```
def operar(f, y):
    return f(y)

print(operar(lambda x: x**2, 4))     # Imprime 16
print(operar(lambda x: x**3, 4))     # Imprime 64

# Por supuesto, si se quiere se les puede acabar
# dando un nombre:
cubo_mas_cuadrado = lambda x: x**3 + x**2

print(operar(cubo_mas_cuadrado, 4)) # Imprime 80
```

- Ojo: **no pueden tener más de una instrucción** (expresión).

Funciones lambda

- Un ejemplo interesante con función `sorted`:

Argumento tipo *lambda-función* para ordenar por clave deseada

```
tuplas_id_asig = [(1918,'TEII'), (1907,'ASO'), (1905,'TDS')]

# Ordenar por 'id' (numéricamente)
print(sorted(tuplas_id_asig, key = lambda tupla: tupla[0]))
# [(1905, 'TDS'), (1907, 'ASO'), (1918, 'TEII')]

# Ordenar por 'asignatura' (alfabéticamente)
print(sorted(tuplas_id_asig, key = lambda tupla: tupla[1]))
# [(1907, 'ASO'), (1905, 'TDS'), (1918, 'TEII')]
```

Parámetros a funciones: posibilidades

Declaración de función muy genérica:

```
def func(p1, p2=7, *args, **kwargs)
```

1. Parámetros obligatorios (ej. `p1`).
2. Parámetros optativos (ej. `p2`).
3. Parámetros posicionales variables (ej. `args`, lista).
4. Parámetros con nombres variables (ej. `kwargs`, diccionario).

Ejemplos de llamadas válidas:

```
func(1)              # p1=1, p2=7
func(1, 2)           # p1=1, p2=2
func(1, p2=2)        # p1=1, p2=2
func(1, 2, 3, 4)     # p1=1, p2=2, args[0]=3, args[1]=4
func(1, a=3, b=4)    # p1=1, p2=7, kwargs['a']=3, kwargs['b']=4
```

Orientación a objetos

- **Clases** con **atributos** (variables de instancia) y **métodos**.
- Pueden variar sobre la marcha (*"on the fly"*).
- En realidad no hay encapsulación ni métodos/atributos privados:
  - ◇ Se espera que el llamador se "porte bien".
- Primer parámetro explícito en los métodos: **`self`** (similar a *this* de Java):
  - ◇ Acceso a métodos/atributos del objeto a través de `self`.
  - ◇ Si no se incluye, entonces es un **método de clase**.
- El constructor es el método especial **`__init__`**
- Existen otros métodos especiales: `__str__`, o todos los relacionados con la sobrecarga de operadores (`__mul__`, `__eq__`, `__lt__`, `__gt__`, `__neg__`, etc.).

Clase de ejemplo

Clase Punto:

```
import math

class Punto:
    def __init__(self, x, y):  # Constructor
        self.x, self.y = x, y

    def distancia_origen(self):
        return math.sqrt(self.x * self.x + self.y * self.y)

    def distancia(self, otro):
        dx, dy = self.x - otro.x, self.y - otro.y
        return math.sqrt(dx * dx + dy * dy)

    def trasladar(self, dx, dy):
        self.x, self.y = self.x + dx, self.y + dy

    def __str__(self):  # Usado para imprimir el objeto
        return "(" + str(self.x) + ", " + str(self.y) + ")"

    def Info():  # Sin parámetro self -> método de clase
        print("Hola, soy la clase Punto")
```

Clase de ejemplo

Uso de la clase Punto:

```
p = Punto(3,-4)                 # Usa método __init__
print(p)                        # Usa método __str__
print(p.distancia_origen())     # 5.0 (no varía el objeto)
p.trasladar(+1,+2)              # Varía el objeto
print(p)                        # (4,-2)
Punto.Info()                    # Método de clase, imprime:
                                #   "Hola, soy la clase Punto"
p.Info()                        # Error (excepción TypeError)
```

- También pueden llamarse a los métodos usando el parámetro self explícitamente como primer parámetro, cualificando sobre el nombre de la clase:

Llamada con uso explícito de self:

```
Punto.trasladar(p,+1,+2)    # Equivale a p.trasladar(+1,+2)
print(p)                    # (14,18)
```

Herencia

- Python soporta **herencia** (incluso **múltiple**).

Herencia en python:

```
class NuevaClase(SuperClase):
    # ... métodos e instancias nuevos ...

class OtraClase(SuperClase1, ..., SuperClaseN):
    # ... métodos e instancias nuevos ...
```

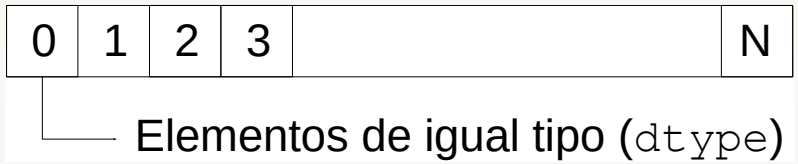
- Si el uso de más de una superclase provoca alguna colisión de atributo/método → se resuelve de izquierda a derecha.
- Si se quiere acceder a un método de una superclase (*overriden*) → se puede usar `super()`.

Ejemplo:

```
class Point3D(Point):    # Punto3D extiende clase Punto
    # Métodos, atributos, etc., nuevos
```

Introducción a Numpy

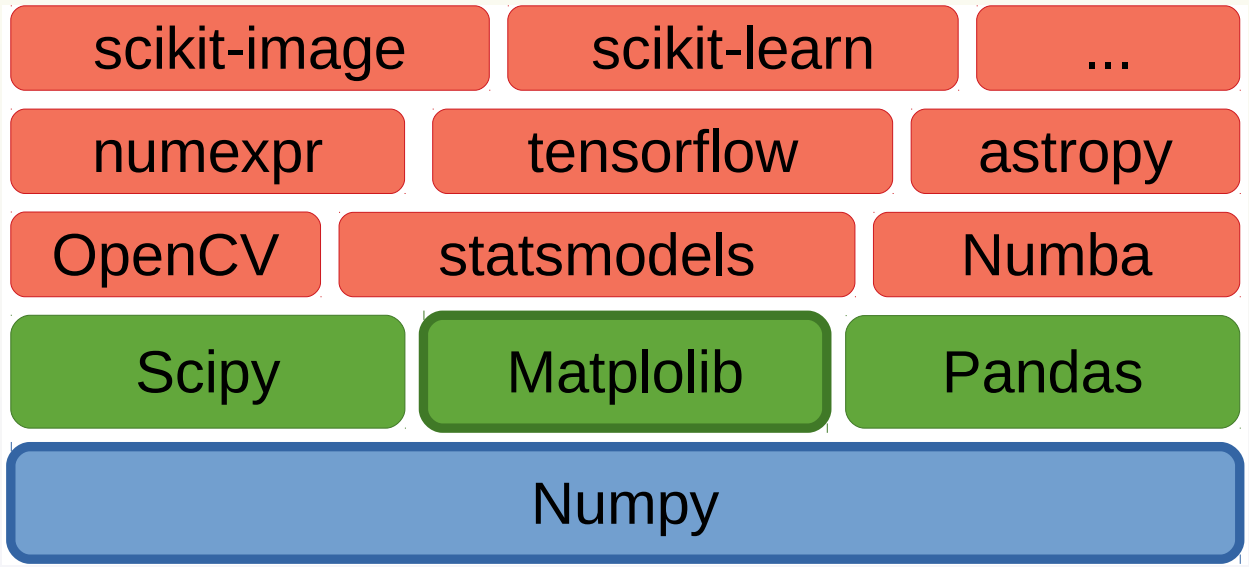
- Extensión de Python, con soporte para grandes **arrays multidimensionales**:
  - ◇ Vectores, matrices, tensores, etc.., con multitud de funciones sobre ellos.
- Basada en el objeto de tipo **`ndarray`**:
  - ◇ Encapsulan arrays n-dimensionales de datos homogéneos:



  - ◇ Implementados en código compilado desde C (no interpretado por Python) $\rightarrow$ Eficiencia.
  - ◇ Usado por la mayoría de paquetes científicos.
  - ◇ Habitualmente creamos objetos `ndarray` mediante funciones como `np.array`, `np.zeros`, `np.ones`, etc.

Ecosistema Numpy

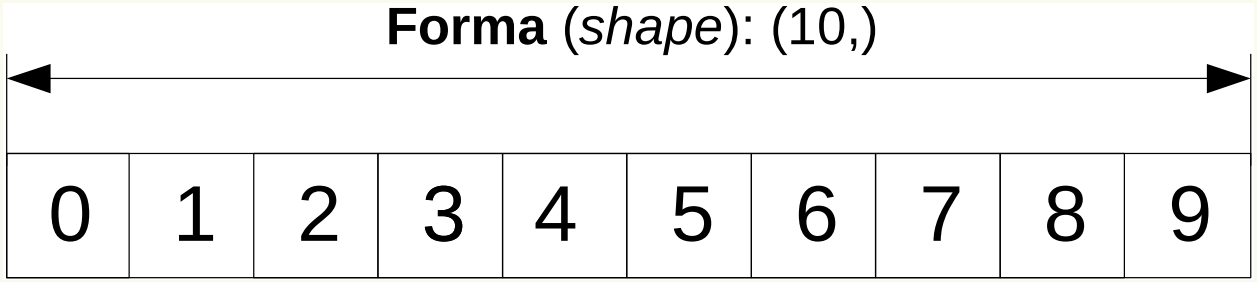
- Una gran cantidad de paquetes de cómputo científico se apoyan en Numpy como librería básica...
- ... Y, sobre ella, Matplotlib es el standard de facto para realizar todo tipo de gráficos.



- Scipy y Pandas son también muy interesantes, pero no las cubriremos en la asignatura.

Forma de un array (*shape*)

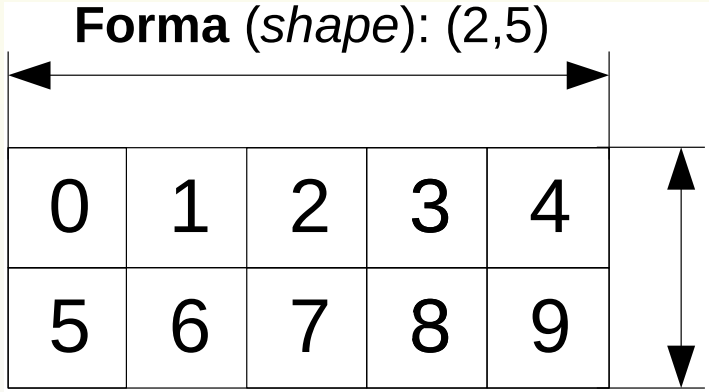
- Los arrays de una dimensión tienen una tupla de dimensión 1 (un sólo elemento) como forma (*shape*):

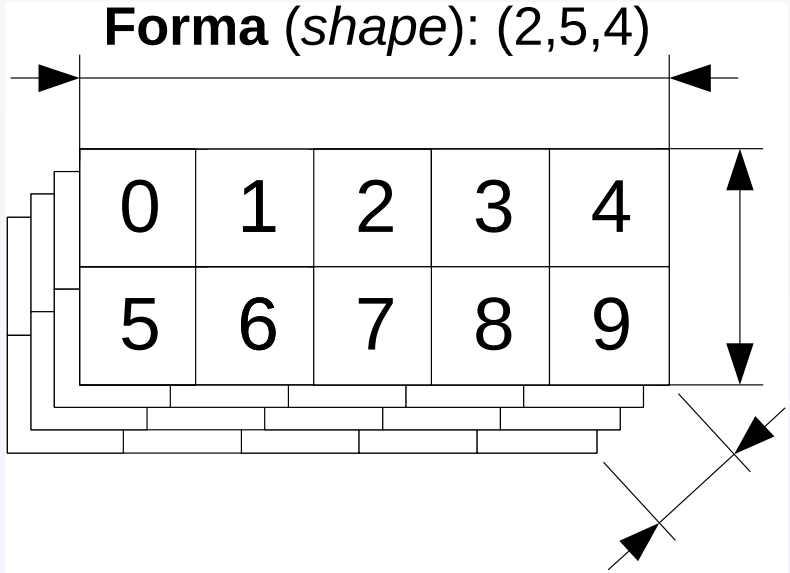


Obsérvese que una tupla de un sólo elemento debe acabar con una coma (10,), no siendo válida en este caso la expresión (10), ya que sería tomada simplemente como un entero (y no una tupla).

Forma de un array (*shape*)

- Los arrays de dos, tres, ... dimensiones van aumentando el tamaño de la tupla de forma acorde:





Creación de arrays y acceso a elementos

Creación a partir de valores concretos:

```
import numpy as np
arr_as_list = [[1.1,2.3],[3.1,-2.7]]
ndarr = np.array(arr_as_list)
ndarr.shape # (2,2)
```

Sin inicialización:

```
np.empty((1,3)) # np.array([[4.9e-324, 9.9e-324, 1.5e-323]])
```

Indicando estructuras predefinidas:

```
identidad5x5 = np.eye(5)
diagonal4x4 = np.diag([.1,.2,.3,.4])
zeros3x4x2 = np.zeros((3,4,2))
unos4x3 = np.ones((4,3))
zeros4x3 = np.zeros_like(unos4x3)
```

Acceso a elementos:

```
A = np.ones((4,5))
A[0,0] += 2
A12 = A[1,2]
prim_fil = A[0,:]
ult_col = A[:,-1]
```

Formas adicionales de creación de arrays

Con secuencias:

```
A = np.arange(0, 10.0, 0.1)        # [0.0,0.1,0.2, ...,9.9]
B = np.linspace(0, 2*np.pi, 100)   # 100 valores en [0,2*pi]
```

Con valores aleatorios:

```
C = np.random.random(size=(3,4))     # Distribución uniforme
D = np.random.normal(loc=0.0, scale=1.0, size=(3,4,5))
                                     # Distribución normal
```

Atributos de un array

Acceso a atributos de instancia:

```
import numpy as np

# Generación de array 1D con posterior reshape:
arr = np.arange(10).reshape((2,5))
arr.ndim    # 2      -> número de dimensiones
arr.shape   # (2,5)  -> forma del array
arr.size    # 10     -> número de elementos
arr.T       # Array transpuesto
arr.dtype   # Tipo de datos de los elementos de array
```

Es posible controlar tipos de datos, comentarios, cabeceras, etc.

Ejemplo de la potencia expresiva de Numpy

Resolución de un sistema de ecuaciones:

```
import numpy as np

# Matriz de coeficientes:
A = np.array([[+2.3, +1.0, -1.1],
              [-4.3, -1.0, -2.7],
              [+3.0, -0.0, +1.1]])

# Vector de términos independientes:
b = np.array([-3.0, -1.0, -0.1])

# Resolución del sistema de ecuaciones Ax = b:
x = np.linalg.solve(A, b)

# Producto matriz vector: operador @
print(A @ x)  # [-3.  -1.  -0.1]

# Influencia de precisión en comprobación de resultados:
print(np.all(A@x==b),np.all(np.isclose(A@x,b))) # False,True
```

Slicing e *indexing*

Múltiples posibilidades de acceso a subarrays:

```
import numpy as np

arr = np.arange(20).reshape((4,5))

# Slicing: Misma idea que listas, pero en todas dimensiones:
arr[0:4, 3:5]  # Todas las filas, últimas dos columnas
arr[:4, 3:5]   # Equivalente
arr[:, 3:5]    # Equivalente

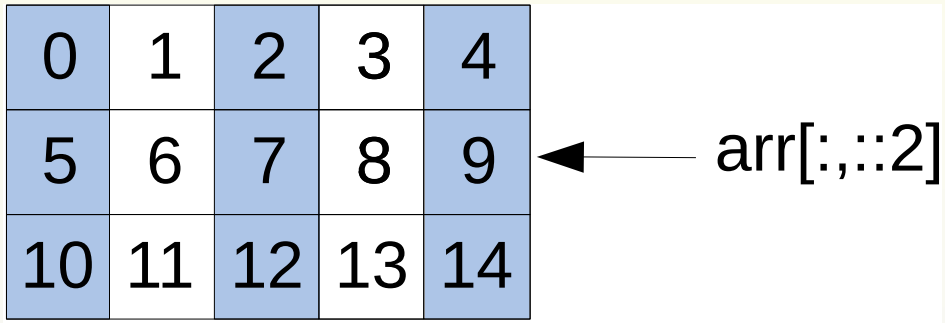
# Índices enteros:
arr[[1,2], :]             # Filas 1 y 2, todas las columnas
arr[np.array([1,2]), :]   # Equivalente

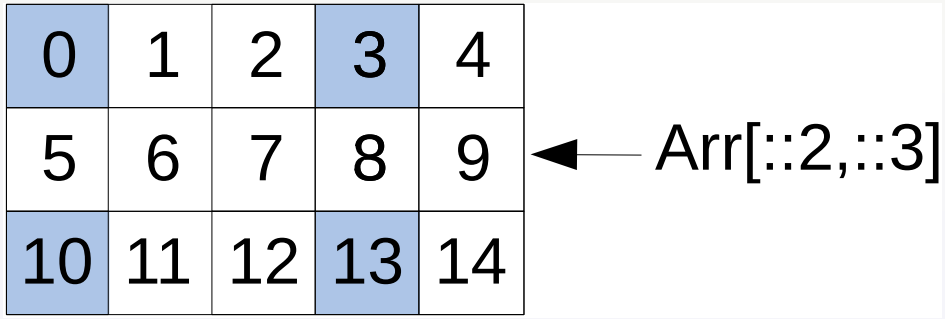
# Índices booleanos:
arr[[False,True,True,False],:]             # Equivalente
arr[np.array([False,True,True,False]),:] # Equivalente
```

¡Particularmente potente es el indexado booleano!

Especificación de *stride* en todas las dimensiones

- Ilustración del funcionamiento del *stride* (análogo al nativo para listas en Python):





Vistas de un array

- Al igual que en Python, las asignaciones **no** realizan copias del array.
- Las operaciones de *slicing* tampoco.
- En lugar de ello, devuelven **vistas** sobre el array original:

Vistas (*views*) de un array:

```
a = np.arange(10)
b = a[3:7]
b # array([3, 4, 5, 6])
b[:] = 0
a # array([0, 1, 3, 0, 0, 0, 0, 7, 8, 9])
b.flags.owndata # False
```

- Las vistas de un array contienen un puntero a los datos originales, aunque posibles diferentes valores de *shape*/*stride*.
- Las vistas siempre tienen el *flag* `flags.owndata = False`.

Tipo básico de un array (*dtype*)

- Los arrays de numpy contienen elementos de un tipo de datos simple.
- Este tipo es accesible a través del atributo `dtype`.
- Algunos atributos interesantes de los objetos de tipo `dtype` son:
  - ◇ `dtype.byteorder` → *Endianness* (orden en memoria).
    '=': nativo '<': little-endian '>': big-endian '|': no aplicable
  - ◇ `dtype.itemsize` → Tamaño de cada elemento.
  - ◇ `dtype.name` → Nombre del tipo.
  - ◇ `dtype.type` → Tipo para crear escalares.

(Aunque hay bastantes más)

Especificación del tipo base del array (*dtype*)

- Los tipos base pueden ser inferidos automáticamente...

Inferencia automática de tipos

```
import numpy as np

a = np.array([1,2,3])
a.dtype  # dtype('int64')
b = np.array([1,2,3,4.567])
b.dtype  # dtype('float64')
```

- ... o bien especificados explícitamente:

Especificación explícita de tipos

```
a = np.array([1,2,3], dtype=np.float32)
a.dtype                          # dtype=float32
a                                # array([ 1., 2., 3.])
b = a.astype(np.float64)  # Conversión de tipo
```

Tipos de datos

NumPy soporta una variedad de tipos numéricos mucho mayor que Python, con tipos primitivos muy similares a los de C.

- Por ejemplo, estos son los basados en tipos discretos:

Tipo numpy	Tipo en C	Descripción
`np.bool`	bool	True o False (almacenado en 1 byte)
`np.byte`	signed char	Dependiente de la plataforma
`np.ubyte`	unsigned char	Dependiente de la plataforma
`np.short`	short	Dependiente de la plataforma
`np.ushort`	unsigned short	Dependiente de la plataforma
`np.intc`	int	Dependiente de la plataforma
`np.uintc`	unsigned int	Dependiente de la plataforma
`np.int_`	long	Dependiente de la plataforma
`np.uint`	unsigned long	Dependiente de la plataforma
`np.longlong`	long long	Dependiente de la plataforma
`np.ulonglong`	unsigned long long	Dependiente de la plataforma

Tipos de datos (cont.)

- Y estos son los basados en tipos "continuos" (punto flotante):

Tipo numpy	Tipo en C	Descripción
`np.half`	-	Float de media precisión: signo, 5 bits exponente, 10 bits mantisa
`np.single`	float	Float de simple precisión: signo, 8 bits exponente, 23 bits mantisa
`np.double`	double	Float de doble precisión: signo, 11 bits exponente, 52 bits mantisa
`np.longdouble`	long double	Float de precisión extendida (dependiente de la plataforma)
`np.csingle`	float complex	Complejos de simple precisión (componentes real e imaginaria float)
`np.cdouble`	double complex	Complejos de doble precisión (componentes real e imaginaria double)
`np.clongdouble`	long double complex	Complejos de precisión extendida (componentes real e imaginaria long double)

Tipos de datos (cont.)

- Alias de tamaños fijos para evitar dependencias de plataforma:

Tipo numpy	Tipo en C	Descripción
`np.int8`	int8_t	Byte [-128, 127]
`np.int16`	int16_t	Entero [-32768, 32767]
`np.int32`	int32_t	Entero [-2147483648, 2147483647]
`np.int64`	int64_t	Entero [-92233720368547758 08, 92233...75807]
`np.uint8`	uint8_t	Entero sin signo [0, 255]
`np.uint16`	uint16_t	Entero sin signo [0, 65535]
`np.uint32`	uint32_t	Entero sin signo [0, 4294967295]
`np.uint64`	uint64_t	Entero sin signo [0, 18446744073709551615]
`np.intp`	intptr_t	Entero para indexar (tamaño ssize_t)
`np.uintp`	uintptr_t	Entero suficiente para un puntero
`np.float32`	float	
`np.float64` / `np.float_`	double	Float de 64 bits (equivalente a Python float)
`np.complex64`	float complex	Complejo con dos floats de 32 bits
`np.complex128` / `np.complex_double`	complex	Complejo con dos floats de 64 bits (equivalente a Python complex)

Operaciones sobre arrays

Operaciones aritméticas/booleanas elemento a elemento:

```
arr1 = np.arange(10).reshape((2,5))
arr2 = np.random.random((2,5))

# Suma, resta, multiplicación, división, ...
arr3 = arr1 + arr2  # Opera elemento a elemento
arr4 = arr1 * arr2  # No confundir con @

# Operaciones inplace:
arr1 += arr4

# Operaciones elemento a elemento:
arr5 = np.sin(arr1)
arr6 = np.sqrt(np.abs(arr2))
```

- Todas las funciones realizan internamente un *dispatch* a código optimizado en C.
- En caso necesario, se hace un *upcast* de los tipos.

Funciones implementadas

- Numpy dispone de una gran variedad de funciones *built-in*:

Tipo	**Funciones**
Aritméticas	`+, -, *, /, ...`
Exponenciales	`exp, exp2, log, log2, log10, power, sqrt, ...`
Trigonométricas	`sin, cos, tan, arcsin, arccos, arctan, arctan2`
Hiperbólicas	Versiones hiperbólicas de las anteriores
Comparación	`<, <=, ==, !=, >=`
Operaciones sobre bits	`&, |, ~, ^, left_shift, right_shift`
Operaciones lógicas	`logical_and, logical_or, logical_not, ...`
Predicados	`isinf, isnan, ...`
Otras	`abs, ceil, floor, round, trunc, mod, sign, ...`

Funciones implementadas

- Métodos sobre arrays:

Tipo	**Funciones**
Predicados	`a.any()`, `a.all()` (para arrays booleanos)
Reducciones	`a.mean()`, `a.argmin()`, `a.argmax()`, `a.trace()`, `a.cumsum()`, `a.cumprod()`
Manipulación	`a.argsort()`, `a.transpose()`, `a.reshape(...)`, `a.ravel()`, `a.fill(...)`, `a.clip(...)`
Complejos	`a.real`, `a.imag`, `a.conj()`

- Algunas funciones adicionales:

Manipulación	`np.hstack()`, `np.vstack()`, `...` (construcción de matrices más grandes a partir de submatrices)
E/S binaria	`np.load()`, `np.save`, `...` (sobre archivos binario .npy (sin comprimir) o .npz (comprimidos)

Entrada/Salida de arrays desde/a ficheros

Escritura y lectura de arrays en ficheros de texto:

```
import numpy as np

# Creación de un array de ejemplo:
arr = np.array([[+2.3, +1.0, -1.1],
                [-4.3, -1.0, -2.7],
                [+3.0, -0.0, +1.1]])

# Salvar en un fichero de texto:
np.savetxt(fname="arr_file.txt", X=arr)

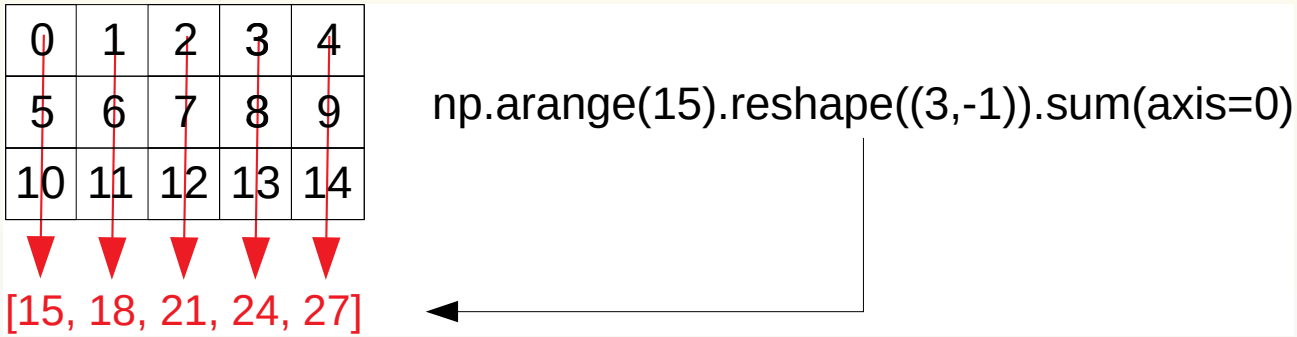
# Volver a cargar desde el fichero:
arr_loaded = np.loadtxt(fname="arr_file.txt")

# Comprobación de corrección:
np.all(arr == arr_loaded) # True
```

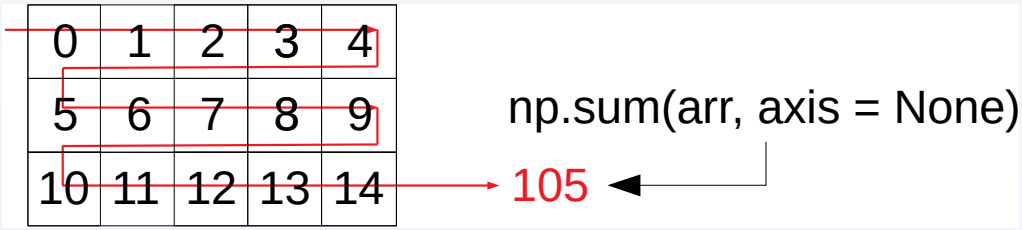
Es posible controlar tipos de datos, comentarios, cabeceras, etc.

Operaciones de reducción

- Las operaciones de reducción (o agregación) (`sum`, `mean`, `max`, `min`, etc.) pueden funcionar sobre sobre un eje concreto (p.e. `Axis = 0`, `Axis = 1`, etc.)...



- ... o bien sobre todos los elementos de un array (es la opción por defecto, usa `Axis = None`):



Broadcasting

- Permite trabajar con arrays de diferentes tamaños:

Broadcasting:

```
import numpy as np

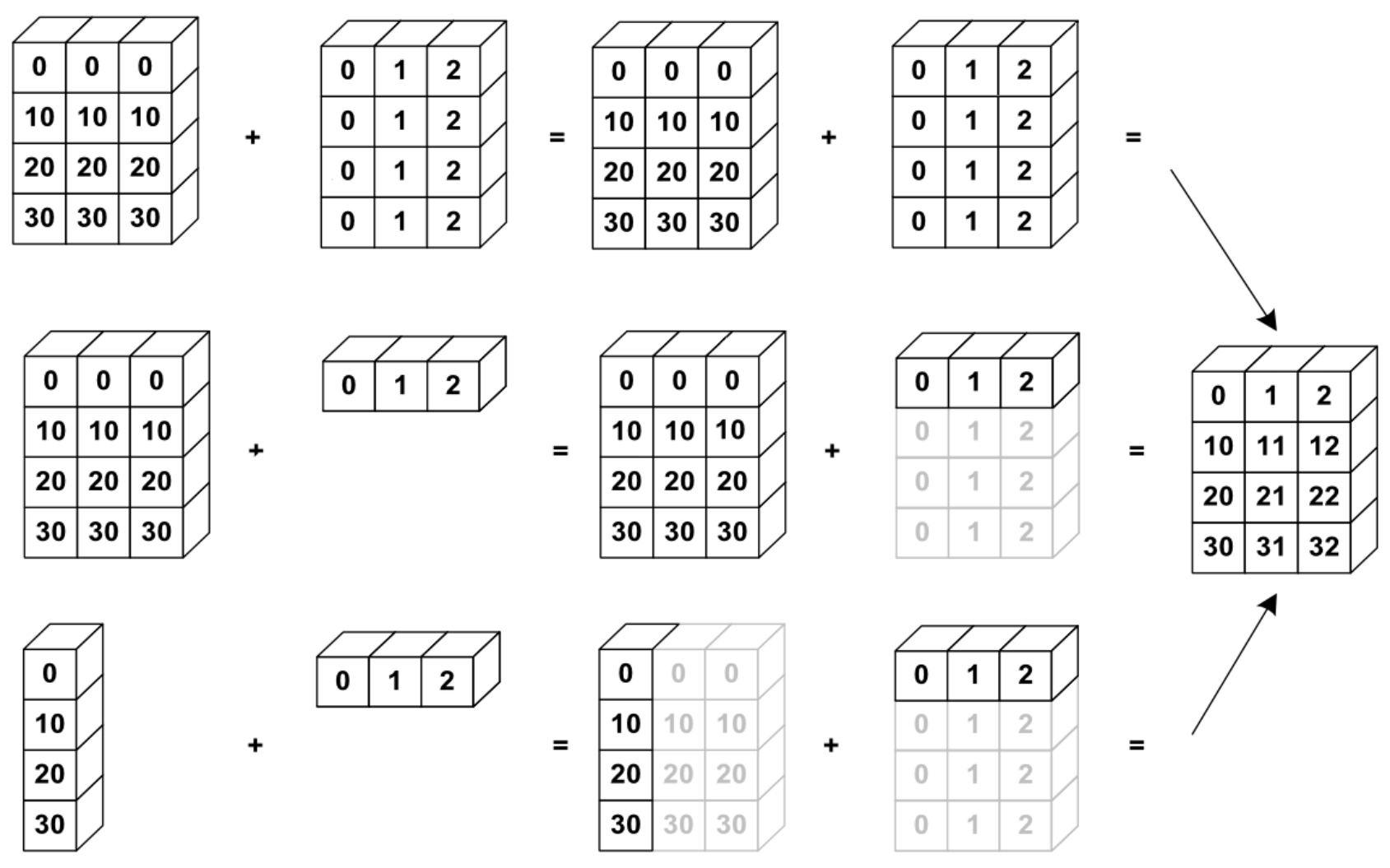
arr = np.arange(20).reshape((4,5))

# Multiplicación por un escalar:
arr *= 5

# Multiplicación de la primera columna por 1, la segunda por
# 2, la tercera por 3, y así sucesivamente:
arr *= np.arange(5)
```

Broadcasting

- Ilustración gráfica del funcionamiento del broadcasting:



Reglas para el *broadcasting*

- Para que una operación pueda realizar bien el *broadcast*, todas las dimensiones "de cola" de ambos arrays deben **ser iguales** o bien **ser 1**.
- Ejemplos:

`A (1d array)`	$\rightarrow$	3
`B (2d array)`	$\rightarrow$	2×3
`Resultado A` *op* `B (2d array)`	$\rightarrow$	2×3
`A (2d array)`	$\rightarrow$	6×1
`B (3d array)`	$\rightarrow$	$1 \times 6 \times 4$
`Resultado A` *op* `B (3d array)`	$\rightarrow$	$1 \times 6 \times 4$
`A (4d array)`	$\rightarrow$	$3 \times 1 \times 6 \times 1$
`B (3d array)`	$\rightarrow$	$2 \times 1 \times 4$
`Resultado A` *op* `B (4d array)`	$\rightarrow$	$3 \times 2 \times 6 \times 4$

Broadcasting: Ajustes de dimensionalidad

- Si las dimensiones no encajan, puede ser útil el uso de `np.newaxis`:

Uso de `np.newaxis`

```
a = np.arange(6).reshape((2, 3))
b = np.array([10, 100])
a * b
# Incorrecto (excepción):
# ValueError: operands could not be broadcast together with
# shapes (2,3) (2)

b[:,np.newaxis].shape  # (2, 1)
a*b[:,np.newaxis]
# Correcto:
# array([[ 0, 10, 20],
#        [300, 400, 500]])
```

Algunos subpaquetes interesantes

- **`numpy.math`**: Librería matemática standard de C.
- **`numpy.linalg`**: Álgebra lineal, con determinantes, inversas, normas, descomposiciones matriciales varias (Cholesky, valores propios, QR, SVD, ...).
- **`numpy.random`**: Generación de números aleatorios, con todo tipo de distrubuciones (normal, uniforme, geométrica, gamma, beta, poisson, etc.).
- **`numpy.fft`**: Transformada rápida de Fourier.
- **`numpy.polynomial`**: Evaluación, álgebra y cálculo eficiente sobre polinomios.

Nota: La librería `scipy`, extensión natural de `numpy`, contiene aún más funcionalidades, p.e., la optimización Simplex que se vio en el bloque 1 de la asignatura.

Matplotlib

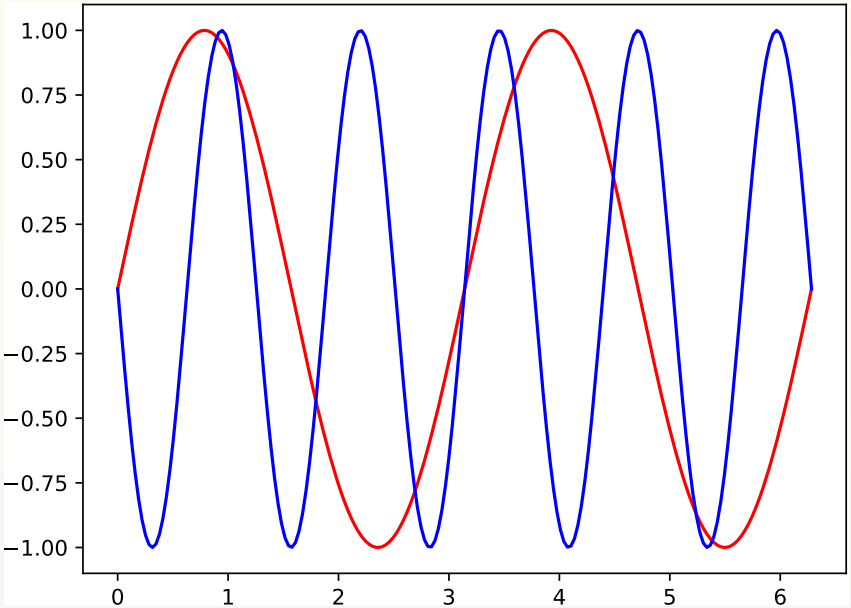
- Librería (estándar de facto) de **gráficos matemáticos** 2D/3D:
  - ◇ Pareja ideal de Numpy.
- Dibujo de funciones, histogramas, diagramas de barras, *scatter plots*, *parametric plots*, ...
- **Muy versátil**:
  - ◇ Prácticamente todo es configurable (programando más)...
  - ◇ ... aunque es sencillo obtener figuras iniciales de calidad.
  - ◇ Permite incluso animaciones e interacción.
- Posibles salidas:
  - ◇ **Arrays de píxeles**
    - · Ventanas de programa
    - · Archivos en formatos jpg, ppm, etc.
  - ◇ **Vectorial**:
    - · Archivos en formatos pdf, svg, etc.

Un primer ejemplo: *Line plots*

La modalidad más habitual (usada para dibujar funciones):

Line plot

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2*np.pi, 200)
y = np.sin(2*x)
z = np.cos(5*x+np.pi/2)
plt.plot(x, y, c='red')
plt.plot(x, z, c='blue')
plt.show()
```



Muchas más alternativas. Por ejemplo:

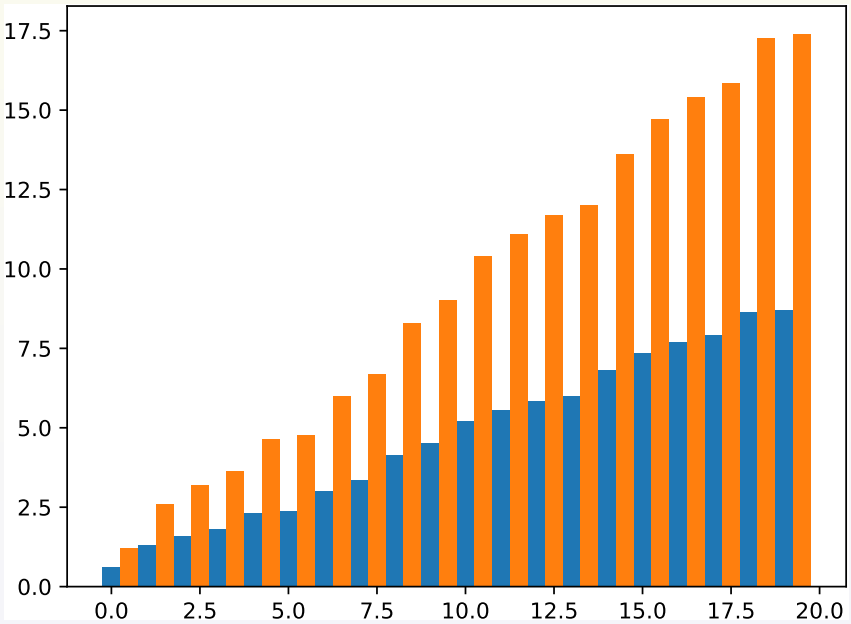
- `plot(x, y, 'bo')` → Usa círculos azules en vez de líneas.
- `plot(y)` → Usa valores de x por defecto (array 0..N-1).
- `plot(y, 'r+')` → Ídem, pero con cruces rojas.

Gráficos de barras: *Bar plots*

Usada principalmente para comparar dos o más series de valores. También para histogramas (aunque para esto hay también una función propia).

Bar plot

```
import numpy as np
import matplotlib.pyplot as plt
x = np.random.random(20)
y = np.cumsum(x)
z = 2*y
plt.bar(np.arange(len(y)), y,
        width=0.5)
plt.bar(np.arange(len(z))+0.5, z,
        width=0.5)
plt.show()
```



Muestras individuales: *Scatter plots*

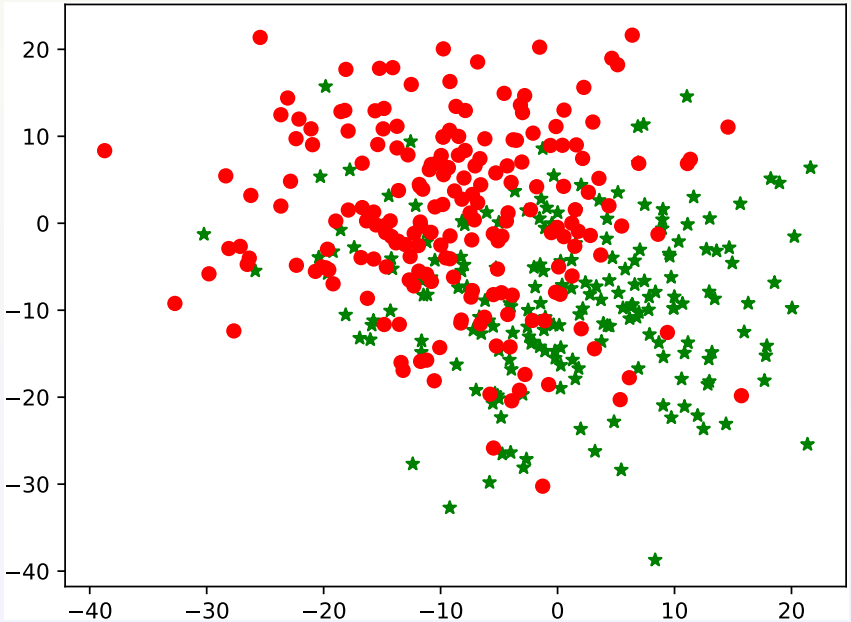
Usados para mostrar una población de puntos 2D, p.e.:

- La posible correlación entre dos variables estadísticas (p.e altura *vs* peso en un conjunto de muestras),
- puntos de posiciones fijas en un mapa,
- la concentración de valores en ciertas zonas, etc.

Scatter plot

```
import numpy as np
import matplotlib.pyplot as plt

x = 10*np.random.randn(200,1)
y = 10*np.random.randn(200,1)-9.0
plt.scatter(x,y,c='g',marker='*')
plt.scatter(y,x,c='r',marker='o')
plt.show()
```

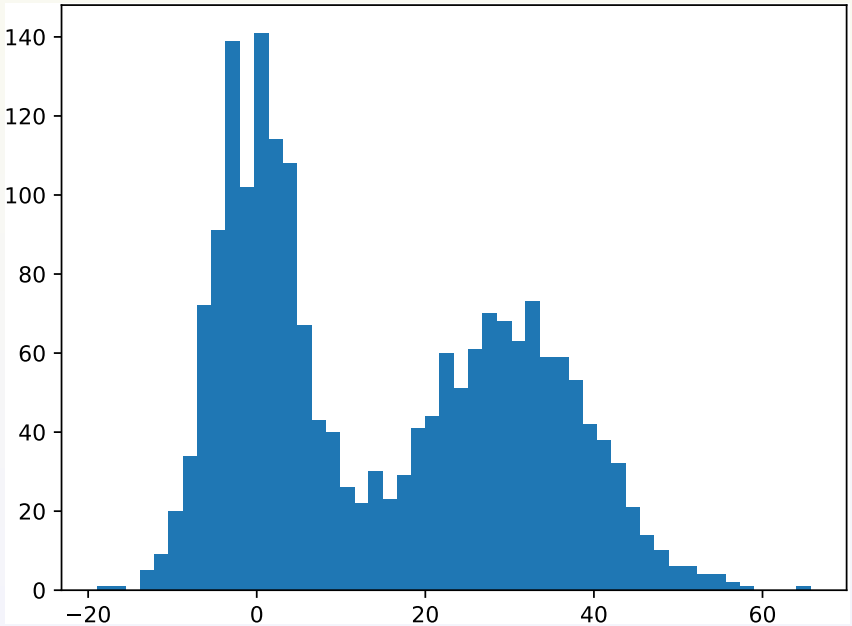


Histogramas: *Histogram plots*

- Usados para mostrar una distribución de probabilidad a partir de muestras.
- La función distribuye implícitamente las muestras en el número de *bins* solicitado, y muestra, para cada *bin*, el número de muestras que caen en el mismo.

Histogram plot

```
import numpy as np
import matplotlib.pyplot as plt
x = 5.0*np.random.randn(2000)
y = 30.0+10.0*np.random.randn(2000)
mask = (np.random.random(2000)>0.5)
plt.hist(np.concatenate(
      (x[mask],y[~mask])), bins=50)
plt.show()
```



Imágenes

Imagen RGB

```
import matplotlib.pyplot as plt

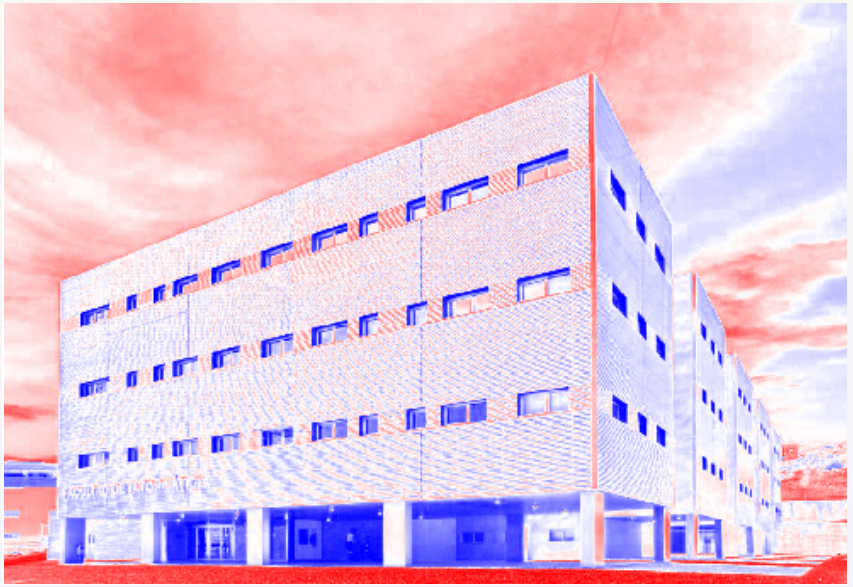
img = plt.imread('fium.jpg')
print(img.shape) # (720,1046,3)
plt.imshow(img)
plt.axis('off')
plt.show()
```



Imagen cambiando mapa de color

```
import matplotlib.pyplot as plt

img = plt.imread('fium.jpg')
plt.set_cmap('bwr')
# Sólo 1er canal (tensor 2D):
plt.imshow(img[:,:,0])
plt.axis('off')
plt.show()
```



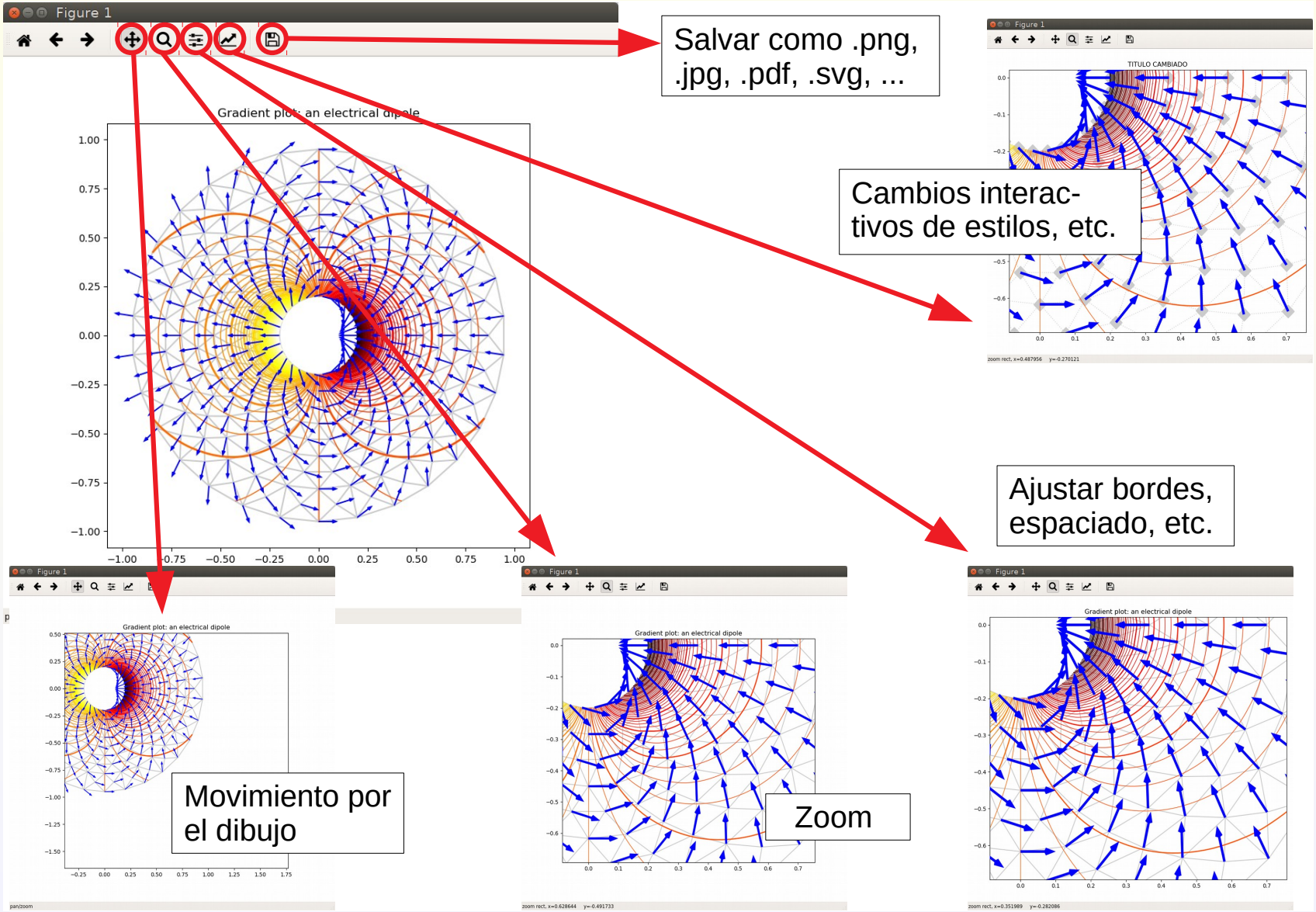
Control de ventanas

- `plt.show()` bloquea el programa hasta que se cierra la figura.
  - ◇ No en intérpretes interactivos (`ipython`, `jupyter`, ...).
- `plt.draw()` actualiza cambios pendientes a una figura (sin mostrarla)...
- ... y `plt.pause(`*`segs`*`)` muestra una figura pendiente, procesando eventos del GUI (salvar archivo, cambiar tamaño de ventana, moverse por el gráfico, etc.)

Muestra figura y espera tecla

```
# Función auxiliar: muestra la figura matplotlib pendiente
# y espera una pulsación de tecla:
def show_plot_and_wait_for_key():
    plt.draw()
    plt.pause(0.01)
    input("<Hit Enter To Close>")
    plt.close()
```

Interfaz gráfico (GUI)



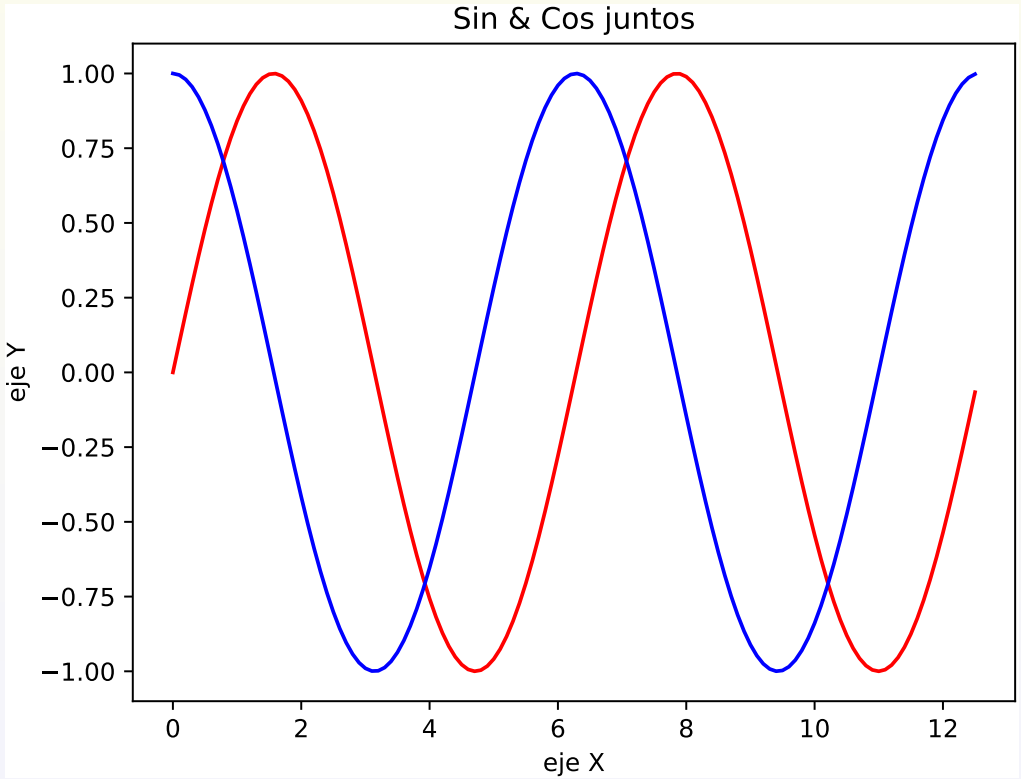
Gráficos múltiples: en un sólo *axes*

- Primera opción: varias gráficas en un sistema de coordenadas **(axes) común**:

Dos *plots* en un sólo *axes*

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0,4*np.pi,0.1)
ss = np.sin(x)
cc = np.cos(x)
plt.plot(x, ss, 'red')
plt.plot(x, cc, 'blue')
plt.xlabel('eje X')
plt.ylabel('eje Y')
plt.title('Sin & Cos juntos')
plt.show()
```



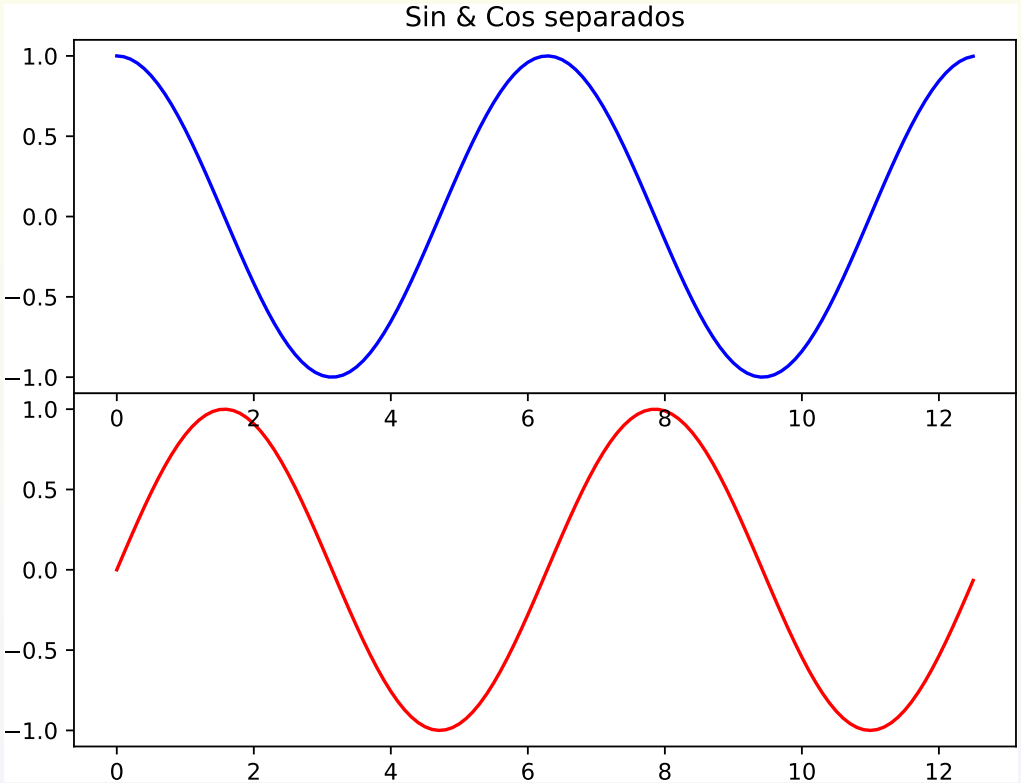
Gráficos múltiples: varios *axes*

- Segunda opción: **varios axes en una figura**:

Dos *axes* en una figura

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0,4*np.pi,0.1)
ss = np.sin(x)
cc = np.cos(x)
plt.axes([0.075,0.05,0.9,0.45])
plt.plot(x, ss, 'red')
plt.axes([0.075,0.50,0.9,0.45])
plt.plot(x, cc, 'blue')
plt.title('Sin & Cos separados')
plt.show()
```



Sintaxis (unidades entre 0.0 y 1.0, dimensiones de la figura):

`plt.axes([x, y, ancho, alto])`

(Dejar espacio para etiquetado, numeración, etc.)

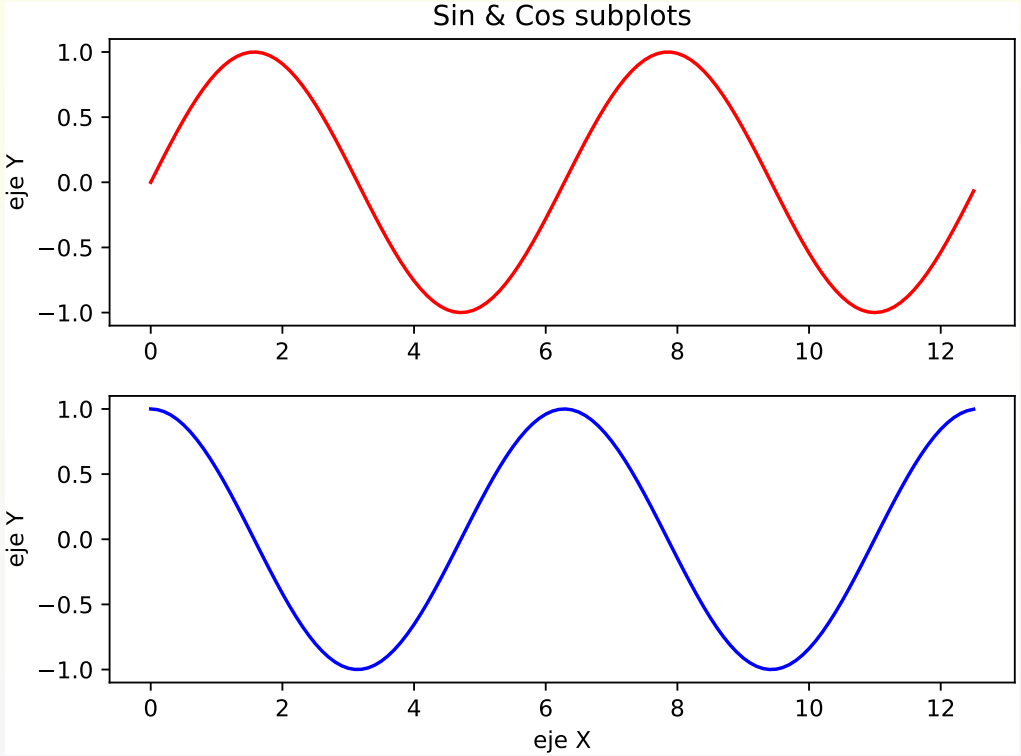
Gráficos múltiples: *subplots*

- Tercera opción: **Usando subplots**:

Uso de *subplots*

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0,4*np.pi,0.1)
ss = np.sin(x)
cc = np.cos(x)
plt.subplot(2, 1, 1)
plt.plot(x, ss, 'red')
plt.ylabel('eje Y')
plt.title('Sin & Cos subplots')
plt.subplot(2, 1, 2)
plt.plot(x, cc, 'blue')
plt.ylabel('eje Y')
plt.xlabel('eje X')
plt.tight_layout()
plt.show()
```



`plt.subplot(nrows, ncols, nsubplt)`

- Orden de subplots: lectoescritura (primero filas, luego columnas)
- Ojo: Partiendo de 1, no de 0.

Control de ejes

- **Extensiones de los ejes**:
  - ◇ `plt.axis([xmin, xmax, ymin, ymax])` para ambos ejes.
  - ◇ `plt.xlim([xmin, xmax])`, `plt.ylim([ymin, ymax])` para ejes por separado.
  - ◇ Pueden usarse tanto tuplas (ej. `xlim((-3, 5))`) como listas (ej. `xlim([-3, 5])`)
  - ◇ Más opciones para `plt.axis(...)`:

`axis('off')`	Desactiva etiquetas y líneas de los ejes
`axis('equal')`	Fuerza igual escalado en ejes X e Y
`axis('square')`	Fuerza un cuadrado (aspect ratio = 1)
`axis('tight')`	Fuerza `xlim()`, `ylim()` tales que se muestran todos los datos

Leyendas

- Las **leyendas** proporcionan un etiquetado cómodo de las series de datos:
  - ◇ Primero hay que etiquetar la serie de datos al hacer el dibujo con un nombre, p.e.:
    - · `plt.plot(x, y, label='Función A')`
    - · `plt.scatter(x, y, marker='o', color='red', label='Puntos B')`
    - · ... (Ídem para `plt.bar()`, etc.) ...
  - ◇ Luego se pone la leyenda con:
    - · `plt.legend(loc='upper right')`
  - ◇ Otras posibilidades: `'lower right'`, `'lower right'`, `'center'`, ... , `'best'`.

Anotaciones

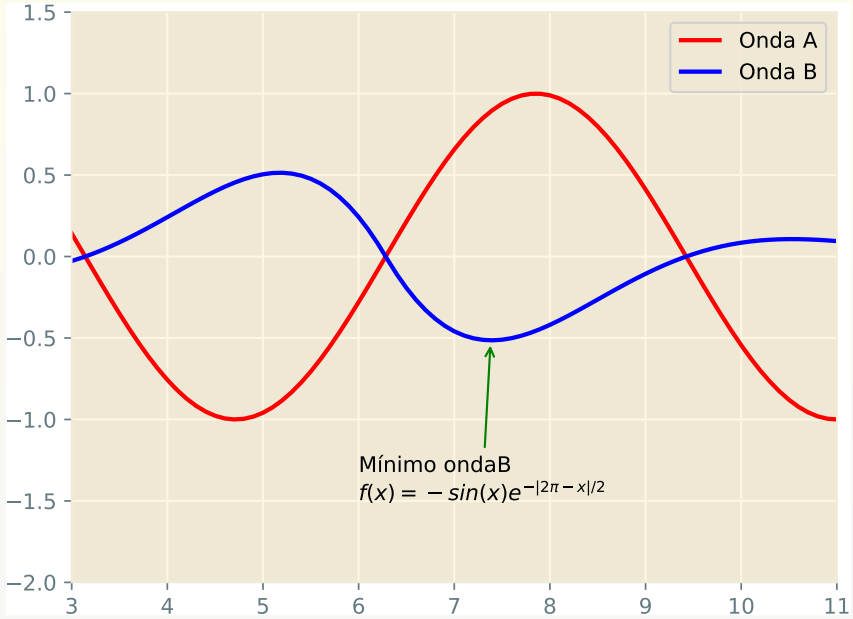
- Las **anotaciones** proporcionan un método para colocar texto arbitrario en un gráfico:
  - ◇ Opcionalmente apuntando a un lugar concreto con una flecha (de color, anchura deseados):
    - · Parámetros `xy` y `xytext`.
  - ◇ Las propiedades de la flecha (color, estilo, anchura, etc.) se pasan en un diccionario (`arrowprops`).
  - ◇ Ejemplo:

```
plt.annotate('Máximo f(x)',
             xy=(6.0, 5.5), xytext=(5.25, 3.0),
             arrowprops={'color': 'green',
                         'arrowstyle': '<->'})
```

Un ejemplo con control de ejes, leyendas y anotaciones

Control de ejes, leyendas y anotaciones

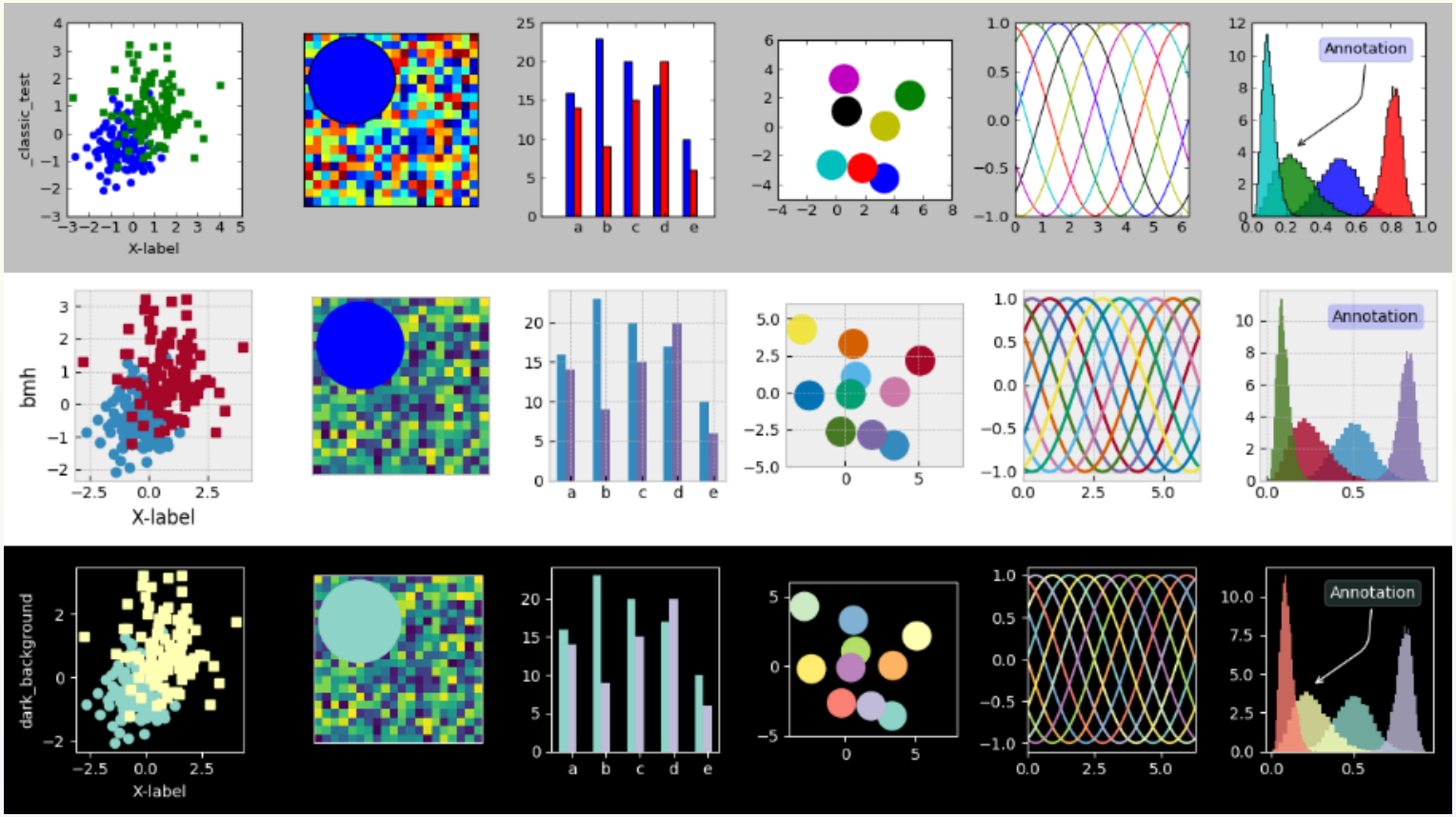
```
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('Solarize_Light2')
x = np.arange(0,4*np.pi,0.1)
A = np.sin(x)
B = -A*np.exp(-np.abs((2*np.pi-x)/2))
plt.plot(x,A,'red',label='Onda A')
plt.plot(x,B,'blue',label='Onda B')
plt.axis([+3.0, 11.0, -2.0, +1.5])
plt.legend(loc='upper right')
plt.annotate('Mínimo ondaB \n'
  '$f(x)=-sin(x)e^{-|2\pi-x|/2}$',
  xy=(7.38,-0.52), xytext=(6.0,-1.5),
  arrowprops={'color':'green',
              'arrowstyle':'->'})
plt.show()
```



Observar el uso posible de LaTeX en anotaciones, ejes, etc.

Estilos

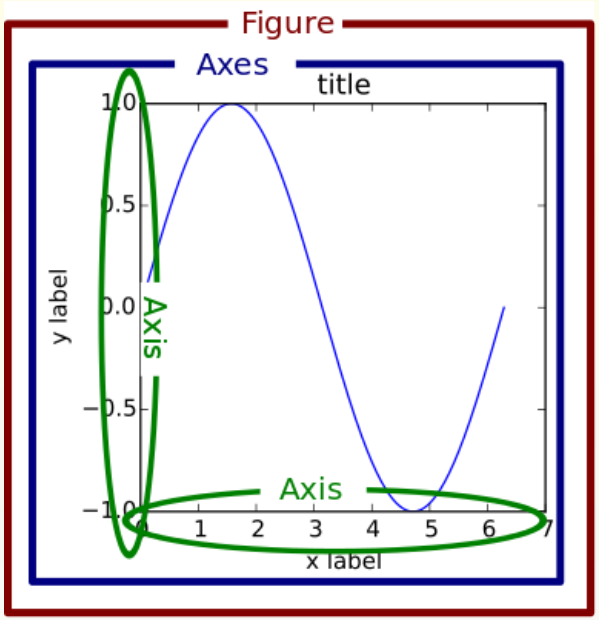
- Para cambiar de estilo, colocar **`plt.style.use('estilo')`** al comienzo del *script* (antes de cualquier otra llamada a `plt`):



Gran variedad de estilos disponibles:

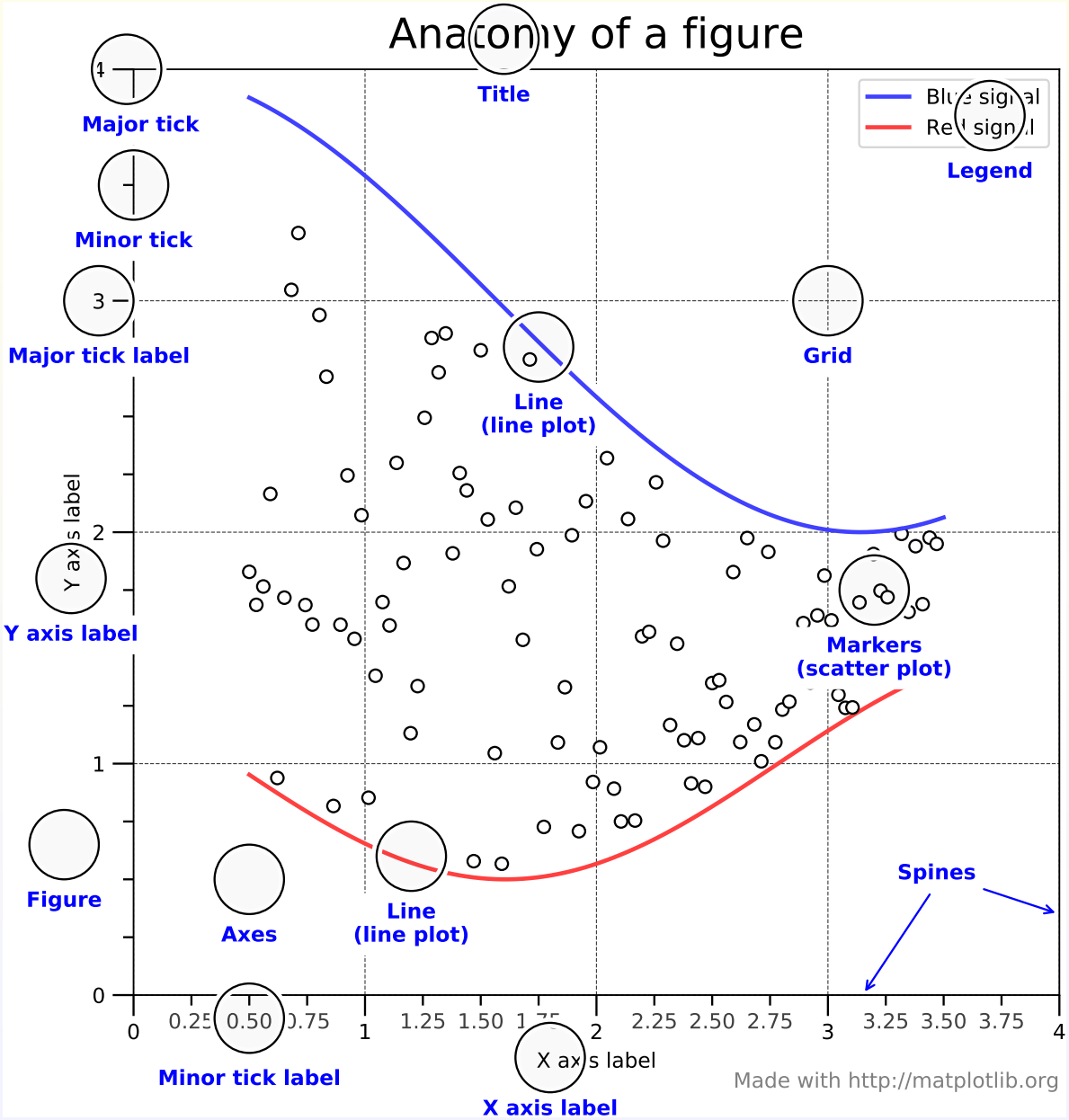
`https://matplotlib.org/gallery/style_sheets/style_sheets_reference.html`

Jerarquía de objetos en `matplotlib`



- Hasta ahora hemos usado sólo métodos `plt.` (`plt.plot()`, `plt.bar()`, `plt.title()`, ...).
- En realidad, un *plot* está formado por una **jerarquía de objetos**.
- El objeto **`Figure`** es un contenedor externo...
- ... que puede contener múltiples objetos **`Axes`** (*plots* individuales) ...
- ... que a su vez contienen objetos **`Axis`** (dos, para el caso típico de gráficos 2D) .
- Al final de la jerarquía hay objetos más pequeños: p.e. `ticks`, `labels` de los ejes, pero también leyendas, anotaciones, etc.

Anatomía de una figura en matplotlib



Acceso a elementos en la jerarquía

- `plt.gcf()` obtiene el actual (más reciente) objeto figura.
- `plt.gca()` ídem, pero para el objeto *axes* actual.

Uso de `plt.gcf()` y `plt.gca()`

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0,4*np.pi,0.1)
ss = np.sin(x)
plt.plot(x, ss)
fig = plt.gcf()
print(type(fig))    # <class 'matplotlib.figure.Figure'>
ax = plt.gca()
print(type(ax))     # <class 'matplotlib.axes._subplots.
                    #          AxesSubplot'>
tick = fig.axes[0].yaxis.get_major_ticks()[0]
print(type(tick))   # <class 'matplotlib.axis.YTick'>
```

Aproximaciones *stateful* vs. *stateless*

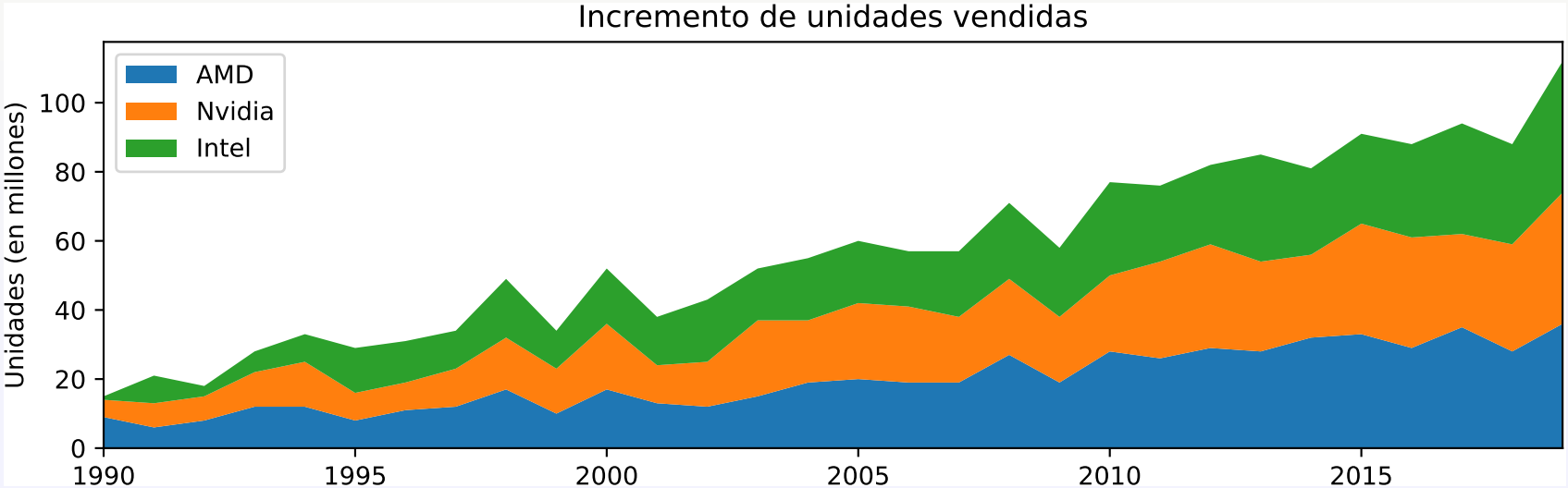
- **Aproximación "con estado":**
  - ◇ Casi todas las funciones del submódulo `pyplot` (todas las vistas hasta ahora), se refieren a la **figura y axes actuales** (o **crean unos nuevos** si ninguno existía).
  - ◇ Las funciones que añaden elementos al dibujo (líneas, barras, texto, imágenes, etc., como `plt.plot()` o `plt.bar()`, p.e.), lo hacen siempre a dicho *axes* actual.
- **Aproximación "sin estado" (mediante objetos)**:
  - ◇ Más *customizable*, útil para gráficos más complejos.
  - ◇ La principal llamada para crear un dibujo bajo esta aproximación OO es **`plt.subplots()`**:

```
fig, ax = plt.subplots() # Una figura con un sólo axes
print(type(ax)) # <class 'AxesSubplot'>
fig, axarr = plt.subplots(nrows=2,ncols=2)
# axarr es ahora un array numpy de objetos tipo Axes:
print(type(axarr[1,0])) # <class 'AxesSubplot'>
```

Ejemplo de uso de OO en matplotlib

Stateless (orientado a objeto) matplotlib

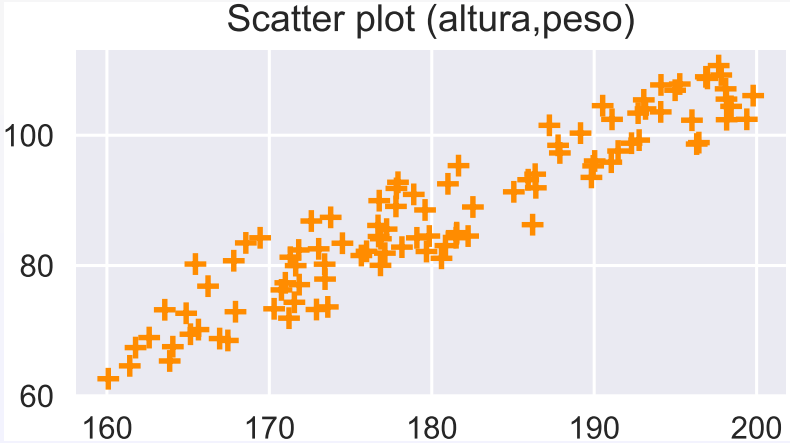
```
intv = np.arange(30)
rnd = np.random.randint(0, 10, size=(3, intv.size))
yrs = 1990 + intv
fig, ax = plt.subplots(figsize=(9, 3))
ax.stackplot(yrs,intv+rnd,labels=['AMD','Nvidia','Intel'])
ax.set_title('Incremento de unidades vendidas')
ax.legend(loc='upper left')
ax.set_ylabel('Unidades (en millones)')
ax.set_xlim(xmin=yrs[0], xmax=yrs[-1])
fig.tight_layout()
plt.show()
```

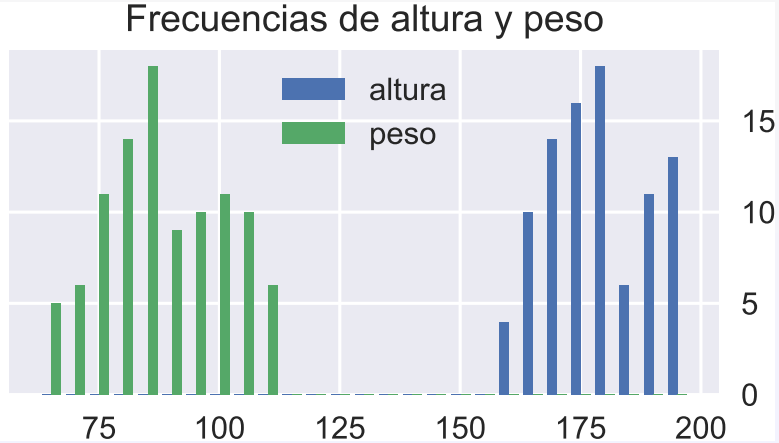


Ejemplo de uso de OO en matplotlib

Otro ejemplo OO, ahora con dos *subplots*

```
plt.style.use('seaborn')
altura = 160 + 40*np.random.random(100)
peso = (altura-100) + 15*np.random.random(100)
data = np.column_stack((altura, peso))
fig, (ax1,ax2) = plt.subplots(nrows=1,ncols=2,figsize=(9,2))
ax1.scatter(x=altura, y=peso, marker='+', c='darkorange')
ax1.set_title('Scatter plot (altura,peso)')
ax2.hist(data,bins=np.arange(data.min(),data.max(),5),
         label=('altura', 'peso'))
ax2.legend(loc='upper center')
ax2.set_title('Frecuencias de altura y peso')
ax2.yaxis.tick_right()
plt.show()
```





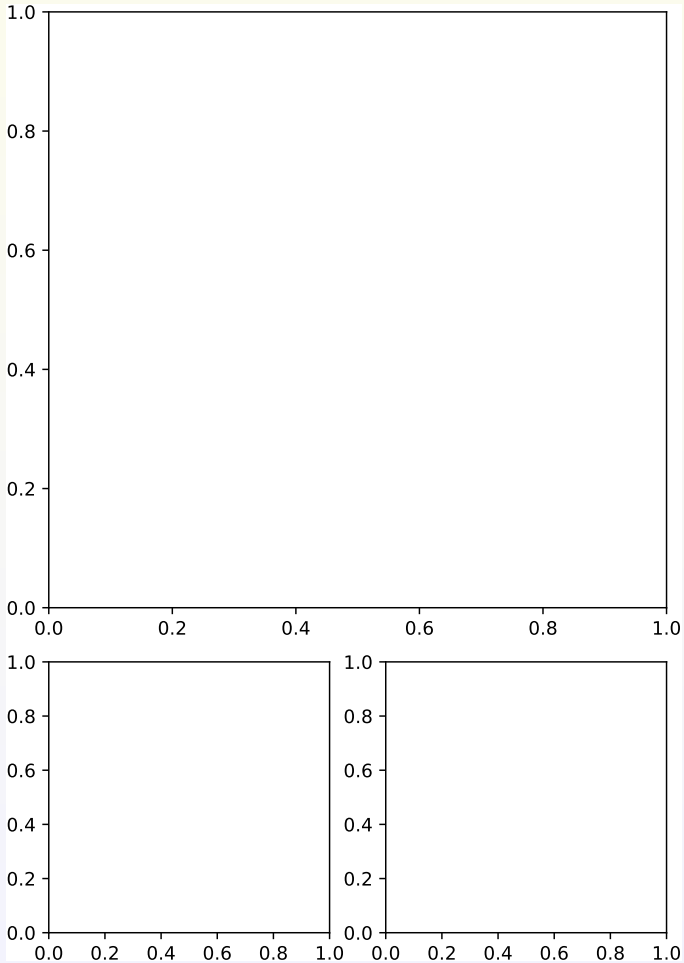
Combinación de gráficos de diferentes tamaños con OO

- Otro uso de la orientación a objetos:
  - ◇ Combinación de gráficos de diferentes tamaños definiendo la disposición (*layout*) deseado sobre una retícula (*grid*):

Layouts ad-hoc con `subplot2grid()`

```
import matplotlib.pyplot as plt
import numpy as np

gridsize=(3, 2)
fig=plt.figure(figsize=(6, 9))
ax1=plt.subplot2grid(gridsize,(0,0),
                     colspan=2,
                     rowspan=2)
ax2=plt.subplot2grid(gridsize,(2,0))
ax3=plt.subplot2grid(gridsize,(2,1))
plt.show()
```



Recomendaciones finales

- Matplotlib es una **librería muy técnica**...,
- ... y por tanto **compleja y con muchos detalles**.
- El uso del **interfaz stateful** facilita mucho las cosas.
- El uso del **interfaz orientado a objetos** es sin embargo más potente y flexible.
- En general, crear un gráfico de calidad requiere fecuentes búsquedas en Internet:
  - ◇ Google (∼4.8M páginas)
  - ◇ Stackoverflow (∼400K páginas)
- No dejar de consultar la **galería de matplotlib** para ver la enorme cantidad de posibilidades ofrecidas por la librería:

`https://matplotlib.org/gallery.html`

Librería `ctypes`

- Permite usar librerías existentes (programadas con otros lenguajes) escribiendo simples **wrappers** de código en Python.
- Trabajaremos con librerías compartidas nativas de Linux (**archivos `.so`**), ...
- ... Si bien exactamente los mismos procedimientos pueden usarse sobre Windows (con archivos `.dll`).
- Cubriremos:
  - ◇ **Carga de librerías** en C
  - ◇ **Llamadas** a funciones en C
  - ◇ Especificación de parámetros y valores devueltos (**prototipos** de funciones)
  - ◇ **Manejo de arrays** (especificados con `numpy`)
  - ◇ **Depuración** con `gdb`

Carga de una librería

- La forma nativa de ctypes para hacerlo es la siguiente:

Carga de una librería compartida (1ª posibilidad)

```
import ctypes
 # Ruta hasta el archivo binario con la librería:
libc = ctypes.CDLL("./libexample.so")
```

- Aunque nosotros usaremos una alternativa (del submódulo correspondiente de la librería numpy):

Carga de una librería compartida (2ª posibilidad)

```
import numpy as np
# Nombre de archivo binario, y ruta hasta el mismo:
libmult = np.ctypeslib.load_library('libmult.so', '../C/')
```

Una vez cargada, la librería se almacena en un **objeto python** que tiene **un método por cada función exportada**.

Llamadas a funciones del módulo

Si no hay argumentos, llamar a una función es trivial:

- La función es simplemente un método del objeto librería recién cargado.
- Y devuelve un valor entero (opción por defecto de C).

Llamada a una función sin argumentos

```
import numpy as np
import ctypes as ct

# Carga de la librería...
libsimple = np.ctypeslib.load_library('libsimple.so', './')

# ... y llamada a la función C exportada por la misma:
counter = libsimple.simple_function()
```

Prototipo de una función

- Para pasar parámetros a una función, y/o recoger valores devueltos de tipo distinto de `int`...
- ... primero hay que hacer saber a python la **signatura** (o prototipo) de la función:

Especificación del prototipo de una función

```
import ctypes as ct
import numpy as np

libmult = np.ctypeslib.load_library('libmult.so', '../C/')
# Especificación de tipos de los parámetros de entrada:
libmult.mult.argtypes = \
 [np.ctypeslib.ndpointer(dtype=np.float32,ndim=1,flags='C'),
  np.ctypeslib.ndpointer(dtype=np.float32,ndim=1,flags='C'),
  ct.c_int, ct.c_float]
# Especificación del tipo devuelto:
libmult.mult.restype = ct.c_int
# (En este caso no era estrictamente necesario, al ser int
#  el tipo devuelto por defecto).
```

Código C original (1/2)

- Éste sería el prototipo de la función en C correspondiente (en archivo `mult.h`):

Prototipo en archivo de cabecera `.h`

```
// Archivo mult.h
// (Archivo de cabecera con prototipos de funciones C)

// Funcion de multiplicacion de un vector de entrada
// por un factor de escala. Trabaja en memoria ya
// reservada por numpy.
int mult(float *vin,      // Vector de entrada
         float *vout,     // Vector de salida
         int size,        // Longitud de los vectores
         float scale);    // Factor de escala
```

Código C original (2/2)

- Y éste el cuerpo del código de la función (en archivo `mult.c`):

Cuerpo de la función en archivo fuente .c

```c
// Archivo mult.c
// (Archivo con cuerpo de codigo de la libreria)
#include <stdlib.h>
#include "mult.h"

// Funcion que multiplica todos los elementos de un
// vector por un factor de escala:
int mult(float *vin, float *vout, int size, float scale)
{
    int i;
    for(i=0;i<size;i++) {
        vout[i] = scale*vin[i];
    }
    return 0;
}
```

Tipos de datos fundamentales

- Lista de principales tipos elementales (primitivos) disponibles:

ctypes type	C type	Python type
c_char	char	Cadena de 1 carácter (1 byte)
c_wchar	wchar_t	Ídem, pero UNICODE
c_byte	char	int
c_ubyte	unsigned char	int
c_short	short	int
c_ushort	unsigned short	int
c_int	int	int
c_uint	unsigned int	int
c_long	long	int
c_ulong	unsigned long	int
c_longlong	long long	int
c_ulonglong	unsigned long long	int
c_float	float	float
c_double	double	float
c_longdouble	long double	float

Interfaz para arrays de Numpy

- La principal forma de pasar un array de Numpy a/desde C (es, decir, en lectura/escritura) es la usada en el ejemplo anterior:

```
np.ctypeslib.ndpointer(dtype=np.float32, ndim=1, flags='C')
```

  - ◇ Hay que indicar el tipo de elementos y el número de dimensiones del tensor.
  - ◇ La memoria contendrá una secuencia contigua de elementos de dicho tipo.
  - ◇ La(s) dimension(es) del array deberá(n) pasarse como parámetros adicionales.
- Esta es la forma más sencilla:
  - ◇ La memoria se crea y se destruye desde python.
- Se pueden emplear también otros mecanismos:
  - ◇ Por ejemplo, creando memoria con `malloc` desde C...
  - ◇ ... pero entonces tambien deberíamos liberarla (con `free`) desde el propio C.

Compilación de librerías

- Y éste es el procedimiento adecuado para compilar la librería con gcc (archivo `Makefile`):

Archivo `Makefile`

```
all: libmult.so.1.0.1 libmult.so

libmult.so.1.0.1: mult.c mult.h
  gcc -Wall -c -fPIC -g mult.c
  gcc -shared -Wl,-soname,libmult.so.1 \
        -o libmult.so.1.0.1 mult.o

libmult.so: libmult.so.1.0.1
  ln -s libmult.so.1.0.1 libmult.so

clean:
  rm -f libmult.so.1.0.1 libmult.so mult.o *~
```

Con dichas opciones, gcc genera el archivo libmult.so.1.0.1 *(ELF 64-bit LSB shared object, x86-64, dynamically linked, with debug info)*.

Función *wrapper* para Python

- Con todo ello, una posible función *wrapper* quedaría así (archivo `multwrap.py`):

Función *wrapper*

```
# Wrapper python para llamar a la función implementada en C:
def mult(arr_in, N):
    # Puesto que usamos el tipo float de 32 bits por defecto
    # de C (en lugar del float de python, que es de 64 bits),
    # hay que convertir primero el array de entrada:
    arr_in_copy = arr_in.astype(np.float32)

    # A continuación hay que crear previamente el array numpy
    # donde se escribirá el vector resultado:
    arr_out = np.zeros_like(arr_in_copy)

    # Ya se puede llamar a la función de la librería:
    libmult.mult(arr_in_copy, arr_out, len(arr_in_copy), N)

    # Y por fin devolver el resultado escrito por la función:
    return arr_out
```

Uso desde Python

- Con todo ello, el uso desde cualquier script de Python convencional es trivial. Por ejemplo (archivo `main.py`):

Uso de la función de wrapper

```
# Uso desde scripts de python convencionales:

from multwrap import mult

out_arr = mult(inp_arr, N)
```

Uso de gdb sobre librerías C cargadas desde Python

- Una librería en C llamada desde Python puede ser también depurada con gdb.
- Eso sí, tiene que haber sido compilada con la opción -g.
- A tener en cuenta:
  - ◇ Lo que se depura es en realidad el intérprete de python:
    $ gdb python
  - ◇ Ya dentro de gdb, se lanza la ejecución pasando como parámetros el nombre del script y los argumentos:
    (gdb) r ./main.py 1.2
  - ◇ Pueden listarse funciones del módulo en C:
    (gdb) l mult
  - ◇ Y por supuesto, a partir de ahí, poner puntos de ruptura, inspeccionar variables, etc.
- En Aula Virtual: ejemplo de sesión de depuración para código de partida proporcionado en prácticas (`sesion-gdb.pdf`).

Recordatorio de comandos gdb más utilizados

Comando	Ejemplo de uso	Descripción
`l (list)`	`l func`	Muestra código fuente.
`r (run)`	`r ./main.py 1.2`	Ejecución (con paso de parámetros).
`b (breakpoint)`	`b 9`	Añade punto de ruptura en un número de línea dado, o un nombre de función.
`c (cont)`	`c`	Continúa hasta siguiente punto de ruptura.
`s (step)`	`s`	Ejecuta línea a línea.
`n (next)`	`n`	Igual que `step`, pero sin (en su caso) meterse en una subrutina.
`p (print)`	`p *vin@50`	Muestra contenidos de variables, arrays, etc.
`wa (watch)`	`watch i`	Para la ejecución cuando cambia una variable (*data breakpoint*).
`info`	`info breakpoints`	Muestra información variada (sobre breakpoints, watchpoints, marco de pila, variables locales...).
`d (delete)`	`d 9`	Elimina breakpoint (recibe el valor que se le dio al `b` correspondiente).
`bt (backtrace)`	`bt`	Muestra traza de llamadas anidadas hasta la función actual.

Referencias I



NumPy developers.
Página principal de la librería
http://www.numpy.org/.



M. Scott.
Una introducción a numpy y scipy
https://engineering.ucsb.edu/~shell/che210d/numpy.pdf.



Anaconda Inc.
Tutorial numpy
https://github.com/continuumio/tutorials/raw/master/intro_to_numpy.pdf.



Google LLC.
Google colab
https://colab.research.google.com.

Referencias II



The Matplotlib Development Team.
Página principal de la librería
http://www.matplotlib.org/.



Real Python.
Python plotting with matplotlib (guide)
https://realpython.com/python-matplotlib-guide/.



Python standard library documentation.
Ctypes: A foreign function library for python
https://docs.python.org/3/library/ctypes.html.



Tom Doeppner.
Gdb cheatsheet
https://cs.brown.edu/courses/cs033/docs/guides/gdb.pdf.

FIN